



FastAPI MASTERY SERIES

FastAPI

The Complete Hands-On Guide

Real-time features, caching, distributed background work, and the observability practices that turn an API into a production system.

● PHASE 3 · ADVANCED

Advanced Pydantic v2

WebSockets

Lifespan Events

Rate Limiting (slowapi)

Caching with Redis

Task Queues (Celery)

Structured Logging

Testing with pytest

Prometheus & Sentry

Hands-on project for this phase: **The PulseChat API** · Built on **FastAPI 0.132+**, **Python 3.10+**, **Pydantic v2**, **redis-py 5.x**, **Celery 5.x**

Verified against the official documentation at fastapi.tiangolo.com · 2026 Edition

From Phase 2 to Phase 3

Phase 2 made TaskFlow real: a database, users, auth, structured errors. Phase 3 changes the questions. Instead of 'does the API work?' the question becomes *'does it work under load, in real time, across multiple servers, and can I prove it?'*

What you should already be comfortable with

- ✓ Dependency Injection, routers, and a layered project structure
- ✓ Async `def` endpoints and async SQLAlchemy
- ✓ Pydantic models with separate input/output schemas, response models, status codes
- ✓ JWT-based authentication and the OAuth2 password flow
- ✓ Custom exception handlers and middleware

If anything on that list is hazy, finish Phase 2 first. Phase 3 builds on every one of them.

YOUR PHASE 3 PROJECT



The PulseChat API

A real-time chat and notifications service. Users join channels, send messages live over WebSockets, get push notifications fanned out by Celery workers, and benefit from Redis-backed caching and rate limiting. By the end of this phase PulseChat is observable, testable, and behaves correctly across multiple server instances.

The mental shift Phase 3 asks of you

1. **Long-lived connections, not just one-shot requests.** WebSockets stay open for minutes or hours; that changes how you manage state.
2. **Coordinated, not isolated.** Multiple workers and instances must agree on things — that is what Redis is for.
3. **Observe, do not guess.** Logs, metrics, and error reporting let you debug a production system you cannot attach a debugger to.
4. **Tests are your safety net.** The features in this phase are subtle; a real test suite is what lets you ship them confidently.

All code uses current best practices: Pydantic v2 validators and computed fields, `redis.asyncio` from `redis-py` (the official replacement for the deprecated `aioredis` package), and modern `lifespan` events.

What you will learn

1 Advanced Pydantic v2

Validators, computed fields, model serialization

2 WebSockets

Bidirectional, low-latency connections for real-time features

3 Lifespan & Startup/Shutdown Events

Properly managing resources across the app lifetime

4 Rate Limiting (slowapi)

Defending your API from abuse and accidental overload

5 Caching & Redis

Fast in-memory state with redis.asyncio

6 Task Queues with Celery

Durable, retryable, distributed background work

7 Logging & Structured Logs

Making your API debuggable in production

8 Testing (pytest, TestClient, mocks)

A real test suite for a real API

9 Monitoring (Prometheus & Sentry)

Metrics, dashboards, and crash reporting

★ Capstone — The Complete PulseChat API

All nine capabilities assembled into one production-shaped service

§ Glossary of Advanced Terms

→ What Comes Next — Phase 4 Preview

Advanced Pydantic v2

In earlier phases Pydantic handled types and basic constraints. Now we use its real power: custom validators that encode business rules, computed fields that compose data on the fly, and serializers that shape output without leaking internals. These are the tools that turn schemas into a robust contract.

Field validators: rules per field

When a built-in constraint isn't enough, `@field_validator` lets you attach custom Python checks to one or more fields.

```
from pydantic import BaseModel, field_validator

class MessageCreate(BaseModel):
    channel: str
    content: str

    @field_validator("content")
    @classmethod
    def no_empty_after_strip(cls, v: str) -> str:
        cleaned = v.strip()
        if not cleaned:
            raise ValueError("Message content cannot be blank")
        return cleaned  # the cleaned value is stored

    @field_validator("channel")
    @classmethod
    def lowercase_channel(cls, v: str) -> str:
        if not v.replace("-", "").isalnum():
            raise ValueError("Channel name must be alphanumeric (with -)")
        return v.lower()
```

A API TERM — `field_validator`

A method decorated with `@field_validator("field_name")` runs after the field's normal type validation. Return the value (possibly transformed) to keep it; raise `ValueError` to reject. Pydantic turns your `ValueError` into a normal FastAPI 422 with the field path included.

A few practical patterns:

You want to...	Use this argument
Validate <i>before</i> the type cast (raw string)	<code>@field_validator("x", mode="before")</code>
Run after the standard validation (default)	<code>@field_validator("x")</code> or <code>mode="after"</code>

You want to...	Use this argument
Validate several fields with one function	<code>@field_validator("a", "b", "c")</code>

Model validators: rules that span fields

Single-field rules can't express things like "end date must be after start date" or "if notify is True, an email must be present." For those, `@model_validator` runs after the whole model is built.

```
from datetime import datetime
from pydantic import BaseModel, model_validator

class ChannelSchedule(BaseModel):
    name: str
    starts_at: datetime
    ends_at: datetime

    @model_validator(mode="after")
    def end_after_start(self) -> "ChannelSchedule":
        if self.ends_at <= self.starts_at:
            raise ValueError("ends_at must be after starts_at")
        return self
```

! WATCH OUT — Return self, not a dict

An `after`-mode model validator is a method on the instance. Modify `self.field = ...` if needed, then `return self`. Returning a dict (a pattern from old Pydantic v1) is not supported in v2.

Computed fields: derived data, automatically

Some output fields are not stored — they are computed from others. A user's full name, an age from a birthdate, a display badge based on roles. `@computed_field` exposes a method as a real field in your output JSON and in the OpenAPI schema.

```
from pydantic import BaseModel, computed_field

class UserPublic(BaseModel):
    id: int
    first_name: str
    last_name: str
    posts_count: int = 0
    is_admin: bool = False

    @computed_field
    @property
    def display_name(self) -> str:
        return f"{self.first_name} {self.last_name}".strip()

    @computed_field
```

```

@property
def badge(self) -> str:
    if self.is_admin:
        return "moderator"
    return "veteran" if self.posts_count > 1000 else "member"

```

Now every JSON response includes `display_name` and `badge`, and the docs document them — even though they aren't stored anywhere. The client never has to compute them.

A API TERM — `computed_field`

A read-only field defined by a method (typically a `@property`) and decorated with `@computed_field`. It appears in serialised output and in the JSON schema, but it cannot be set on input. Perfect for derived display data.

Serializers: shaping output without breaking the type

Sometimes you want to keep the rich type internally but emit something simpler in JSON — a datetime as a Unix timestamp, a Decimal as a string. `@field_serializer` customises how a field is rendered, without changing the underlying type.

```

from datetime import datetime
from pydantic import BaseModel, field_serializer

class Message(BaseModel):
    id: int
    content: str
    created_at: datetime

    @field_serializer("created_at")
    def to_unix(self, v: datetime) -> int:
        return int(v.timestamp())

```

Strict vs lax validation

By default Pydantic happily coerces — the string `"5"` becomes the integer `5`. For external boundaries where you want exact types, mark fields strict:

```

from typing import Annotated
from pydantic import BaseModel, Field, StrictInt

class Pricing(BaseModel):
    cents: StrictInt  # only real ints accepted
    multiplier: Annotated[float, Field(strict=True)]

```

★ **PRO TIP — Default v2 settings are sensible**

v2's defaults are deliberately stricter than v1. `True` no longer coerces silently to `1` in many places, and stringly-typed numbers go through documented coercion. If you're migrating from v1, read the migration page — the current FastAPI docs have a dedicated section for it.

Reusable settings with pydantic-settings

One specialised use of Pydantic worth knowing: configuration. `pydantic-settings` loads environment variables into a typed, validated `Settings` object. This is the recommended way to handle config in FastAPI projects.

```
# pip install pydantic-settings
# app/core/config.py
from functools import lru_cache
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", case_sensitive=True)

    DATABASE_URL: str
    REDIS_URL: str = "redis://localhost:6379/0"
    SECRET_KEY: str
    ACCESS_TOKEN_EXPIRE_MINUTES: int = 60

    @lru_cache
    def get_settings() -> Settings:
        return Settings()
```

Inject `get_settings` as a dependency anywhere you need config — the `@lru_cache` guarantees one instance per process.

▶ **TRY IT YOURSELF — Wire PulseChat's first schemas**

Create `MessageCreate` with the validators above, and `UserPublic` with the computed fields. Test rejection in `/docs` with a whitespace-only message — confirm the 422 names the field exactly.

WebSockets

A WebSocket is a long-lived, bidirectional connection between client and server. Where HTTP is "ask, get answered, disconnect," WebSockets stay open and let either side push messages at any time. It is what powers chat, live dashboards, multiplayer interaction, and collaborative editing.

How WebSockets differ from HTTP

	HTTP	WebSocket
Lifetime	One request, one response	Open for minutes or hours
Initiator	Client only	Either side can send
Format	Per-request JSON/form	Stream of messages (text or binary)
State	Stateless	Per-connection state, naturally

A API TERM — WebSocket

A protocol that starts as an HTTP request, then upgrades to a persistent two-way channel. Once upgraded, client and server exchange discrete messages with very low overhead. FastAPI supports them natively through Starlette.

Your first WebSocket endpoint

The decorator is `@app.websocket`; you receive a `WebSocket` object and use it in an async loop.

```
from fastapi import FastAPI, WebSocket, WebSocketDisconnect

app = FastAPI()

@app.websocket("/ws/echo")
async def echo(ws: WebSocket):
    await ws.accept()
    try:
        while True:
            msg = await ws.receive_text()
            await ws.send_text(f"echo: {msg}")
    except WebSocketDisconnect:
        # the client closed the connection
        pass
```

Read that loop carefully — it is the shape of every WebSocket handler:

1. `ws.accept()` — agree to the upgrade. Until you call this, no messages flow.

2. An async loop that `await` s incoming messages.
3. Wrap the loop in `try/except WebSocketDisconnect` to react to clients going away.

! WATCH OUT — Forgetting `accept()` is the #1 beginner trap

If your handler returns before calling `ws.accept()`, the client just sees a closed connection with no useful error. The very first line of every WebSocket handler should be `await ws.accept()` (possibly behind a quick auth check that raises a normal HTTP error before the accept).

Sending JSON

Text is fine for echo demos. Real apps send structured messages. WebSockets have helpers for JSON, and you can plug Pydantic in for validation just like with HTTP.

```
from pydantic import BaseModel, ValidationError

class ChatIn(BaseModel):
    channel: str
    content: str

class ChatOut(BaseModel):
    user_id: int
    channel: str
    content: str

@app.websocket("/ws/chat")
async def chat(ws: WebSocket):
    await ws.accept()
    try:
        while True:
            raw = await ws.receive_json()
            try:
                msg = ChatIn.model_validate(raw)
            except ValidationError as e:
                await ws.send_json({"error": "invalid", "issues": e.errors()})
                continue
            out = ChatOut(user_id=1, channel=msg.channel, content=msg.content)
            await ws.send_json(out.model_dump())
    except WebSocketDisconnect:
        pass
```

Authenticating a WebSocket

WebSockets don't fit cleanly into the OAuth2 password flow because browsers can't set custom headers on the upgrade request. The standard workaround: pass the token as a query parameter, validate it in the handler, close the socket if it fails.

```

from fastapi import status, Query
from app.core.security import decode_token

@app.websocket("/ws/chat")
async def chat(ws: WebSocket, token: str = Query(...)):
    try:
        user_id = decode_token(token)          # raises if bad
    except Exception:
        await ws.close(code=status.WS_1008_POLICY_VIOLATION)
        return
    await ws.accept()
    ...

```

A API TERM — Close codes

Each WebSocket close carries a numeric code explaining why. `1000` is a normal close, `1008` is policy violation (auth failed), `1011` is server error. Use them deliberately — clients can react to them.

Broadcasting to multiple clients

The first real architectural step: when User A sends a message to a channel, every other client in that channel should see it. Hold the active connections somewhere and iterate.

```

class ChannelManager:
    def __init__(self) -> None:
        self.rooms: dict[str, set[WebSocket]] = {}

    async def join(self, channel: str, ws: WebSocket) -> None:
        await ws.accept()
        self.rooms.setdefault(channel, set()).add(ws)

    def leave(self, channel: str, ws: WebSocket) -> None:
        self.rooms.get(channel, set()).discard(ws)

    async def broadcast(self, channel: str, message: dict) -> None:
        dead = []
        for ws in self.rooms.get(channel, set()):
            try:
                await ws.send_json(message)
            except Exception:
                dead.append(ws)
        for ws in dead:                                     # clean up dropped sockets
            self.leave(channel, ws)

manager = ChannelManager()

@app.websocket("/ws/chat/{channel}")
async def chat(ws: WebSocket, channel: str):
    await manager.join(channel, ws)
    try:

```

```

while True:
    msg = await ws.receive_json()
    await manager.broadcast(channel, msg)
except WebSocketDisconnect:
    manager.leave(channel, ws)

```

The multi-instance problem (foreshadowing Redis)

This in-memory manager works perfectly — for one server process. The moment you run two copies behind a load balancer (the normal production setup), users connected to instance A cannot see messages from users on instance B. The fix is to **push every message through a shared channel** — that is exactly what Redis Pub/Sub is for, and we build it in Chapter 5.

? WHY IT MATTERS — Local state is the wrong default for real-time

The instinct is to keep connection state in process memory. It works in development and breaks on the first production scale-out. Designing for multiple instances from day one — even if you run one today — saves a painful refactor later.

Testing it now

FastAPI's `TestClient` supports WebSockets out of the box (we go deep in Chapter 8). A quick smoke test:

```

from fastapi.testclient import TestClient

with TestClient(app).websocket_connect("/ws/chat/general") as ws:
    ws.send_json({"channel": "general", "content": "hello"})
    msg = ws.receive_json()
    assert msg["content"] == "hello"

```

▶ TRY IT YOURSELF — Build the chat broadcast

Add the `ChannelManager`, the WebSocket route, and a minimal HTML page in `static/` that connects to it. Open two browser tabs to `/ws/chat/general`, type in one — the other receives. You now have a real-time chat server.

Lifespan & Startup/Shutdown Events

An app's "request handlers" are not the whole story. Connection pools must be opened, background loops must be launched, metrics exporters must be registered, and all of it must be cleanly shut down.

`lifespan` is the mechanism for that.

The shape of a lifespan

A API TERM — Lifespan context manager

An `async` function decorated with `@asynccontextmanager` that runs *before* `yield` on startup and *after* on shutdown. Pass it to `FastAPI(lifespan=...)`. It replaces the deprecated `@app.on_event("startup"/"shutdown")` decorators.

```
from contextlib import asynccontextmanager
from fastapi import FastAPI

@asynccontextmanager
async def lifespan(app: FastAPI):
    # ---- startup: build resources -----
    print("starting up")
    yield
    # ---- shutdown: tear them down -----
    print("shutting down")

app = FastAPI(lifespan=lifespan)
```

Whatever you put before `yield` runs once when the server starts; whatever is after runs once when it stops — including on `Ctrl+C`, a graceful redeploy, or a container being killed.

Sharing resources with your endpoints

The classic use: open a connection pool once and let every request reuse it. Attach the pool to `app.state` and read it back via the `Request` object in a dependency.

```
from redis import asyncio as aioredis
from fastapi import FastAPI, Request

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.redis = aioredis.from_url(
        "redis://localhost:6379/0",
        decode_responses=True,
    )
```

```

try:
    yield
finally:
    await app.state.redis.aclose()

app = FastAPI(lifespan=lifespan)

def get_redis(request: Request) -> aioredis.Redis:
    return request.app.state.redis

```

A API TERM — `app.state`

A free-form namespace on the `FastAPI` instance for per-app shared data — typically resources created once at startup. Inside an endpoint or dependency, `request.app.state.thing` retrieves it. Not to be confused with `request.state` (Chapter 8 of Phase 2), which is per-request.

Lifespan vs the old `on_event` decorators

You will see plenty of old code using `@app.on_event("startup")`. It still works, but it is deprecated and missing the key feature: a single function that *owns* setup and teardown together. That matters when something fails halfway through startup — with lifespan, the `try/finally` ensures every resource opened so far is cleaned up.

	Lifespan (modern)	<code>@app.on_event</code> (legacy)
Style	One <code>@asynccontextmanager</code> function	Two separate decorated functions
Setup/teardown coupling	Same function — naturally paired	Separate — easy to drift apart
Error handling on partial startup	<code>try/finally</code>	Manual
Status	Recommended	Deprecated

What belongs in a lifespan

Things that need exactly one setup at process start, one teardown at process end:

- Database engine / connection pool (created once, shared by all sessions)
- Redis client and pub/sub subscriber tasks
- Prometheus metric registry initialisation (Chapter 9)
- Sentry SDK initialisation (Chapter 9)
- Pre-loading large in-memory data: ML models, GeoIP databases
- Starting long-running asyncio tasks (a metrics flusher, a Redis subscriber loop)

! WATCH OUT — Do not run schema migrations here

It is tempting to call `alembic upgrade head` from lifespan. Don't. Migrations are a deployment step, not a runtime step. If you scale to N replicas and they all try to migrate at once, you get races and partial-state databases. Run migrations as a separate command before your service rolls out — Phase 4 shows the CI/CD pattern.

Background tasks living for the app lifetime

Sometimes you need an async task to run continuously alongside your endpoints — a Redis subscriber, a metrics flusher, a heartbeat. Spawn it during startup, cancel it during shutdown.

```
import asyncio

async def redis_subscriber(app: FastAPI) -> None:
    pubsub = app.state.redis.pubsub()
    await pubsub.subscribe("pulse:messages")
    async for raw in pubsub.listen():
        if raw["type"] == "message":
            ... # fan out to local WebSocket clients (Ch. 5)

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.redis = aioredis.from_url("redis://localhost:6379/0",
                                         decode_responses=True)

    task = asyncio.create_task(redis_subscriber(app))
    try:
        yield
    finally:
        task.cancel()
        try:
            await task
        except asyncio.CancelledError:
            pass
        await app.state.redis.aclose()
```

★ PRO TIP — Always await the cancelled task

`task.cancel()` only requests cancellation. The task may still be midway through a database write. Awaiting it (and catching `CancelledError`) lets the task finish or unwind cleanly. Skipping this risks corrupt state on shutdown.

Testing the lifespan

Until you use the app in a `with` block (which `TestClient` does internally), `lifespan` does not run. For async tests with `httpx.AsyncClient`, wrap the call so startup actually fires. The current FastAPI docs include a dedicated section ("Testing Events: lifespan and startup - shutdown") with the recommended patterns.

 TRY IT YOURSELF — Open a Redis pool in PulseChat

Add the Redis-pool lifespan above to PulseChat's `main.py`. Make a simple `GET /ping/redis` that does `await redis.ping()` and returns the result. Stop the server with Ctrl+C and confirm the shutdown message prints — the cleanup ran.

Rate Limiting (slowapi)

A public endpoint with no limits is an invitation: bots will scrape it, sloppy clients will flood it, and one bug can take your service down. Rate limiting caps how often a client can call you — per IP, per user, per route — and is the single cheapest defence you can add.

What rate limiting really is

A API TERM — Rate limiting

A policy that allows N actions per time window from a given client. When the limit is exceeded, the server returns HTTP `429 Too Many Requests`. Limits protect against abuse, accidental loops, runaway scrapers, and brute-force attacks against your login endpoint.

The simplest mental model: every client gets a bucket of tokens. Each request takes one. The bucket refills slowly. Out of tokens? You're rate-limited.

slowapi: the standard library for FastAPI

`slowapi` wraps the well-tested `limits` package and integrates cleanly with FastAPI. It works in-memory for development and uses Redis as the backend in production so limits are shared across instances.

```
pip install slowapi
```

`app/core/limiter.py`

```
# app/core/limiter.py
from slowapi import Limiter
from slowapi.util import get_remote_address

limiter = Limiter(
    key_func=get_remote_address,           # one bucket per IP
    storage_uri="redis://localhost:6379/0", # shared across instances
    default_limits=["100/minute"],        # safety net
)
```

A API TERM — key_func

The function that decides 'who is this request?'. `get_remote_address` uses the client IP — good default. For authenticated APIs, switch to a function that returns the user id, so each user gets their own bucket regardless of network.

Wiring it into the app

app/main.py

```
# app/main.py
from fastapi import FastAPI, Request
from slowapi.errors import RateLimitExceeded
from slowapi.middleware import SlowAPIMiddleware
from app.core.limiter import limiter

app = FastAPI(title="PulseChat API")
app.state.limiter = limiter

# Convert RateLimitExceeded into a 429 response
@app.exception_handler(RateLimitExceeded)
async def rate_limit_handler(request: Request, exc: RateLimitExceeded):
    from fastapi.responses import JSONResponse
    return JSONResponse(
        status_code=429,
        content={"error": "rate_limited", "detail": str(exc.detail)},
        headers={"Retry-After": str(exc.retry_after or 60)},
    )

app.add_middleware(SlowAPIMiddleware)
```

Applying limits per endpoint

Limits live as a decorator on the route. The string syntax is intuitive — slowapi parses things like `"5/minute"`, `"1000/hour"`, `"10/second"`.

```
@router.post("/auth/login")
@limiter.limit("5/minute")                                # brute-force protection
async def login(request: Request, ...):
    ...

@router.post("/messages")
@limiter.limit("60/minute")                                # generous, prevents flood
async def send_message(request: Request, ...):
    ...

@router.post("/upload")
@limiter.limit("10/hour")                                  # expensive op, tight cap
async def upload(request: Request, ...):
    ...
```

! WATCH OUT — The Request parameter is required

slowapi reads the client identity from the `Request` object, so any endpoint with `@limiter.limit` must include `request: Request` as a parameter. Forget it and you get a confusing startup error — not a runtime one.

Limiting by user, not IP

For authenticated APIs you usually want one bucket per user. Define a custom key function:

```
def user_key_func(request: Request) -> str:
    # populated by your auth dependency or middleware
    user_id = getattr(request.state, "user_id", None)
    if user_id is not None:
        return f"user:{user_id}"
    return get_remote_address(request)      # fall back to IP

limiter = Limiter(key_func=user_key_func, storage_uri="redis://...")
```

Choosing sensible limits

Endpoint type	Suggested limit	Why
Login / signup / password reset	5–10 per minute	Slow brute-force attacks
Public reads (list, search)	60–120 per minute	Generous for normal use, hostile to scrapers
Sending messages / posts	30–60 per minute	Real users don't exceed this
Heavy operations (file uploads, reports)	10–30 per hour	Expensive — protect resources
WebSocket connect	5–10 per minute per IP	Stop connection-churn attacks

★ PRO TIP — Layered limits beat one big rule

A single global limit is blunt. The professional pattern: a generous default at the app level (your safety net), tighter per-endpoint limits on sensitive routes, and a different `key_func` for authenticated vs anonymous traffic. Multiple cheap decorators are easier to tune than one giant rule.

The 429 response and Retry-After

When a client trips a limit, the standard response is `HTTP 429 Too Many Requests` with a `Retry-After` header telling them how many seconds to wait. Well-behaved clients honour it; the rest can wait. The exception handler above sets both correctly.

 TRY IT YOURSELF — Lock down PulseChat's login

Add `slowapi` with a Redis backend (the same Redis you opened in Chapter 3). Put `@limiter.limit("5/minute")` on the login endpoint. Send six bad login attempts in a row — the sixth returns 429 with a `Retry-After`. You just blocked credential-stuffing.

Caching & Redis

Redis is an in-memory key-value store that does one job spectacularly well: fast shared state. We will use it for two distinct purposes — caching expensive results and coordinating real-time messages across server instances.

Two uses, one tool

Use	What it solves	Where in PulseChat
Caching	Avoid repeating expensive work (DB queries, API calls)	Channel metadata, user profiles
Pub/Sub	Broadcast messages between processes	Fan out chat messages across instances

The current correct package

! WATCH OUT — aioredis is deprecated — use redis-py

You will see many tutorials use `aioredis`. That package was officially merged into `redis-py` in 4.2 and now lives at `redis.asyncio`. The standalone `aioredis` package is unmaintained. Install only `redis`; do not pip install aioredis.

```
pip install redis
```

```
# Import style most projects use:
from redis import asyncio as aioredis    # alias for ergonomics – same module
# or simply:
import redis.asyncio as redis aio
```

Connection pool via lifespan

Open one connection pool when the app starts, share it everywhere, close it on shutdown — the pattern from Chapter 3 applied to Redis.

app/db/redis.py

```
# app/db/redis.py
from redis import asyncio as aioredis
from typing import Annotated
from fastapi import Depends, Request

async def get_redis(request: Request) -> aioredis.Redis:
    return request.app.state.redis
```

```
RedisDep = Annotated[aioredis.Redis, Depends(get_redis)]
```

app/main.py – lifespan

```
# app/main.py
from contextlib import asynccontextmanager
from redis import asyncio as aioredis
from app.core.config import get_settings

@asynccontextmanager
async def lifespan(app):
    settings = get_settings()
    app.state.redis = aioredis.from_url(
        settings.REDIS_URL,
        decode_responses=True,          # return str, not bytes
        max_connections=20,
    )
    try:
        yield
    finally:
        await app.state.redis.aclose()
```

A API TERM — decode_responses

Redis stores bytes by default and that is what the client returns. Setting `decode_responses=True` tells the client to decode UTF-8 for you — comfortable for JSON-shaped data. If you store binary blobs, leave it off.

A simple read-through cache

The pattern: check the cache; if empty, hit the database and write the result back with a TTL.

```
import json
from app.crud import channel as crud_channel
from app.schemas.channel import ChannelPublic

async def get_channel(redis, db, slug: str) -> ChannelPublic:
    key = f"channel:{slug}"
    cached = await redis.get(key)
    if cached is not None:
        return ChannelPublic.model_validate_json(cached)

    channel = await crud_channel.get_by_slug(db, slug)
    payload = ChannelPublic.model_validate(channel).model_dump_json()
    await redis.set(key, payload, ex=300)      # expire in 5 minutes
    return ChannelPublic.model_validate_json(payload)
```

redis method	Use
<code>get(k)</code> / <code>set(k, v, ex=...)</code>	String values with TTL
<code>delete(k)</code>	Invalidate a cached entry
<code>incr(k)</code> / <code>expire(k, s)</code>	Counters with auto-expiry
<code>hset</code> / <code>hget</code> / <code>hgetall</code>	Compact hashes (e.g. per-user state)
<code>publish</code> / <code>subscribe</code>	Pub/Sub messaging across processes

Cache invalidation: the hard part

The classic joke: there are only two hard things in computer science — cache invalidation, naming things, and off-by-one errors. The rule for invalidation is simple to *state*: whenever the underlying data changes, drop or update the cache key.

```
async def update_channel(redis, db, slug, data):
    channel = await crud_channel.update(db, slug, data)
    await redis.delete(f"channel:{slug}")      # bust the cache
    return channel
```

★ PRO TIP — Short TTLs forgive missed invalidations

A 5-minute TTL means a missed `delete` causes 5 minutes of stale data, not eternity. Start with conservative TTLs (60s–5m) and only extend them once you trust your invalidation paths. Cache miss is fine; cache lying is the danger.

Pub/Sub: the real-time backbone

Remember the multi-instance problem from Chapter 2's chat? Two server instances, each with its own in-memory `ChannelManager`, cannot see each other's clients. Redis Pub/Sub solves it: when any instance receives a message, it *publishes* to Redis; every instance is *subscribed* and forwards to its local WebSocket clients.

```
import asyncio, json
from fastapi import WebSocket

class ChatHub:
    def __init__(self, redis):
        self.redis = redis
        self.local: dict[str, set[WebSocket]] = {}

    async def join(self, channel: str, ws: WebSocket) -> None:
        await ws.accept()
        self.local.setdefault(channel, set()).add(ws)

    def leave(self, channel: str, ws: WebSocket) -> None:
```

```

        self.local.get(channel, set()).discard(ws)

    async def publish(self, channel: str, message: dict) -> None:
        # send to Redis; every instance will receive and broadcast locally
        await self.redis.publish(f"chan:{channel}", json.dumps(message))

    async def listen_forever(self) -> None:
        pubsub = self.redis.pubsub()
        await pubsub.psubscribe("chan:*")
        async for raw in pubsub.listen():
            if raw["type"] != "pmessage":
                continue
            channel = raw["channel"].split(":", 1)[1]
            message = json.loads(raw["data"])
            for ws in list(self.local.get(channel, ())):
                try:
                    await ws.send_json(message)
                except Exception:
                    self.leave(channel, ws)

```

Hook it up in the lifespan — start the subscriber loop on startup and cancel it on shutdown:

```

@asynccontextmanager
async def lifespan(app):
    app.state.redis = aioredis.from_url(REDIS_URL, decode_responses=True)
    app.state.chat_hub = ChatHub(app.state.redis)
    task = asyncio.create_task(app.state.chat_hub.listen_forever())
    try:
        yield
    finally:
        task.cancel()
        try: await task
        except asyncio.CancelledError: pass
        await app.state.redis.aclose()

```

❓ WHY IT MATTERS — Pub/Sub is what makes scale-out possible

Without it, doubling your servers halves the chat experience: half the users miss half the messages. With it, any number of API instances behave like a single one, and you can roll, scale, and deploy without anyone noticing.

🎮 TRY IT YOURSELF — Make PulseChat multi-instance ready

Wire up the `ChatHub`, refactor your WebSocket route to call `hub.publish` instead of broadcasting locally. Start two copies of the app on ports 8000 and 8001, connect a browser to each, and watch a message typed in one appear in the other — the round trip went through Redis.

Task Queues with Celery

Phase 2's background tasks were fine for short, fire-and-forget work — but they vanish when the server restarts and they cannot retry. Real systems push important work to a **task queue**: separate worker processes that pull jobs from a broker, run them reliably, and survive restarts. Celery is the standard Python choice.

How a task queue is wired

A API TERM — Broker, worker, task

A **broker** (Redis or RabbitMQ) is a message bus where jobs are queued. A **worker** is a separate process that pulls jobs and runs them. A **task** is a Python function decorated to be queueable — your code calls `.delay(...)` instead of running it, and a worker picks it up later.



The API enqueues; the worker runs. Two separate processes, one shared broker, no shared memory.

Install

```
pip install "celery[redis]"
```

Configure Celery

app/worker/celery_app.py

```
# app/worker/celery_app.py
from celery import Celery
from app.core.config import get_settings

settings = get_settings()

celery_app = Celery(
    "pulsechat",
    broker=settings.REDIS_URL,
    backend=settings.REDIS_URL,      # store results (optional but useful)
    include=["app.worker.tasks"],   # modules where tasks live
)

celery_app.conf.update(
    task_acks_late=True,             # ack only after success – retries on crash
)
```



```

task_reject_on_worker_lost=True,
task_default_retry_delay=10,
task_time_limit=60,           # hard kill after 60s
task_soft_time_limit=45,      # graceful shutdown at 45s
)

```

A API TERM — Result backend

Optional storage for task results, configured by `backend=`. With it, your code can later check 'did task X finish?' or 'what did it return?'. For fire-and-forget jobs (emails, audits), skip the backend — it keeps Redis smaller and the system simpler.

Define some tasks

app/worker/tasks.py

```

# app/worker/tasks.py
from celery.utils.log import get_task_logger
from app.worker.celery_app import celery_app

log = get_task_logger(__name__)

@celery_app.task(name="pulse.send_email", bind=True,
                 autoretry_for=(Exception,), retry_backoff=True, max_retries=5)
def send_email(self, to: str, subject: str, body: str) -> None:
    log.info("Sending email to %s", to)
    # ...actually send via SMTP / SES / SendGrid here...

@celery_app.task(name="pulse.fanout_push", bind=True,
                 autoretry_for=(Exception,), retry_backoff=True, max_retries=3)
def fanout_push(self, channel: str, message_id: int) -> None:
    # look up subscribers, queue an individual push per device, etc.
    ...

```

The key options to understand:

<code>@celery_app.task</code> option	Effect
<code>name="..."</code>	Stable name across renames — set this explicitly
<code>bind=True</code>	First arg is the task instance — needed for self.retry, etc.
<code>autoretry_for=(Exception,)</code>	Retry automatically on these exceptions
<code>retry_backoff=True</code>	Exponential delay between retries
<code>max_retries=5</code>	Give up after this many failures
<code>task_time_limit</code>	Kill a task that runs longer than this

Enqueue from FastAPI

Inside your route, replace direct work with `.delay(...)`. The route returns immediately; the worker picks up the job.

```
from app.worker.tasks import send_email, fanout_push

@router.post("/messages", response_model=MessagePublic, status_code=201)
async def post_message(data: MessageCreate, user: CurrentUser, db: SessionDep):
    msg = await crud_msg.create(db, data, owner_id=user.id)
    fanout_push.delay(data.channel, msg.id)          # async fan-out
    return msg

@router.post("/users/{user_id}/welcome")
async def welcome_user(user_id: int, db: SessionDep):
    user = await crud_user.get(db, user_id)
    send_email.delay(user.email, "Welcome!", render_welcome(user))
    return {"status": "queued"}
```

! WATCH OUT — Pass primitives, not ORM objects

Whatever you pass to `.delay(...)` gets serialised (usually JSON) and sent through the broker. SQLAlchemy model instances do not survive that trip. Pass the id and fetch the object inside the task with a fresh DB session.

Run a worker

```
celery -A app.worker.celery_app worker --loglevel=info
```

That command launches a worker process that connects to Redis, listens for tasks, and runs them. In production you run multiple workers (one or more per machine) and a process manager (systemd, supervisord, Kubernetes) restarts crashed ones.

Scheduled tasks: Celery Beat

Need a task to run every hour? Use Celery Beat — a scheduler process that enqueues tasks on a cron-like schedule.

```
celery_app.conf.beat_schedule = {
    "expire-old-channels": {
        "task": "pulse.expire_old_channels",
        "schedule": 3600.0,          # every hour
    },
    "daily-digest": {
        "task": "pulse.daily_digest",
        "schedule": crontab(hour=8, minute=0),  # 8:00 every day
    },
}
```

```
celery -A app.worker.celery_app beat --loglevel=info
```

API TERM — Beat

Celery Beat is a separate scheduler process — run exactly one — that enqueues tasks at set times. The workers run them. Together: Beat is the cron, workers are the executors.

BackgroundTasks vs Celery

When the work is...	Pick	Why
A few quick lines, doesn't need to survive restart	BackgroundTasks (Ph. 2)	Zero infrastructure
Important, must run eventually, can be retried	Celery	Durable, retryable
Heavy CPU work (image, ML inference)	Celery	Worker can be scaled independently
Scheduled (every hour, every night)	Celery + Beat	Beat handles the schedule
Triggered by many APIs across a fleet	Celery	Shared queue

TRY IT YOURSELF — Push notifications for PulseChat

Set up Celery + Redis, define a `fanout_push` task, and call `.delay(...)` from the message endpoint. Run the API in one terminal and the worker in another. Post a message and watch the task run in the worker's log a moment later — completely independent of the API process.

Logging & Structured Logs

When something breaks at 3 a.m. and you cannot attach a debugger, logs are all you have. Plain text logs are barely better than nothing in a distributed system; **structured** logs — machine-readable JSON with consistent fields — are the difference between 'I can't find anything' and 'here's the exact request id of the failure.'

Structured vs free-form

Free-form log	Structured log (JSON)
User 42 failed to post in #general	<code>{"event": "post_failed", "user_id": 42, "channel": "general"}</code>
Has to be grep'd and parsed by humans	Indexed, queried, aggregated by log tools
Field names drift, no schema	Same field names everywhere, schema enforced

A API TERM — Structured logging

Emitting each log line as a machine-readable record (typically JSON) with consistent fields like `timestamp`, `level`, `event`, `request_id`, `user_id`. Tools like Datadog, Loki, ELK, and Cloud Logging can then filter and aggregate them. Free-form messages become a needle in a haystack at scale.

Choosing a library

Two solid choices in 2026:

	stdlib logging + JSON formatter	structlog
What it is	Python's built-in module + a formatter	A purpose-built structured library
Setup	Minimal	Slightly more, with helpful defaults
Fit for FastAPI	Good — works with uvicorn out of the box	Excellent — context binding shines

We will use `structlog` for the project — its context binding feature pairs beautifully with FastAPI's request scope.

Configuring structlog

```
pip install structlog
```

```
app/core/logging.py
```

```
# app/core/logging.py
import logging, sys, structlog

def configure_logging(level: str = "INFO") -> None:
    # Route stdlib logging (uvicorn, sqlalchemy) into the same processors
    logging.basicConfig(
        format="%(message)s", stream=sys.stdout, level=level,
    )

    structlog.configure(
        processors=[
            structlog.contextvars.merge_contextvars,
            structlog.processors.add_log_level,
            structlog.processors.TimeStamper(fmt="iso"),
            structlog.processors.dict_tracebacks,
            structlog.processors.JSONRenderer(),
        ],
        wrapper_class=structlog.make_filtering_bound_logger(
            logging.getLevelName(level)),
        cache_logger_on_first_use=True,
    )

log = structlog.get_logger()
```

Call `configure_logging()` once at startup (in the lifespan). Every `log.info(...)` now emits a structured JSON line.

```
log.info("message_posted", user_id=42, channel="general", message_id=901)
# {"event": "message_posted", "user_id": 42, "channel": "general", "message_id": 901,
#  "level": "info", "timestamp": "2026-05-31T09:14:55.012Z"}
```

Per-request context: the killer feature

The Chapter 8 middleware from Phase 2 already gave every request a unique `request_id`. Bind it to structlog's context and every log line emitted during that request automatically carries it.

```
import structlog
from fastapi import Request

@app.middleware("http")
async def logging_context(request: Request, call_next):
    structlog.contextvars.clear_contextvars()
    structlog.contextvars.bind_contextvars(
        request_id=getattr(request.state, "request_id", "-"),
        method=request.method,
        path=request.url.path,
    )
    log.info("request_started")
    try:
```

```

    response = await call_next(request)
    finally:
        log.info("request_finished",
                status=getattr(response, "status_code", 500))
    return response

```

? WHY IT MATTERS — Context turns logs into a story

Without context, a 500 error log says only that something broke. With `request_id` bound, you can pull every line from that one request — the DB query that ran, the Redis hit, the user id, the path — in chronological order. That is how you debug a real outage.

Bind more as you learn it

Inside an endpoint or dependency, add to the binding as soon as you know more:

```

@router.post("/messages")
async def post_message(data: MessageCreate, user: CurrentUser, ...):
    structlog.contextvars.bind_contextvars(user_id=user.id, channel=data.channel)
    msg = await crud_msg.create(db, data, owner_id=user.id)
    log.info("message_posted", message_id=msg.id)
    return msg

```

The auth dependency can bind `user_id` for you; the channel router can bind `channel`. By the time anything interesting happens, every log line carries the full breadcrumb trail.

Log levels and what each is for

Level	When	Example
DEBUG	Verbose detail useful during development only	raw SQL, payloads
INFO	Normal events worth recording	request started, message posted
WARNING	Unexpected but recoverable	fallback path taken, deprecated endpoint
ERROR	A request or task failed	DB timeout, downstream 500
CRITICAL	Service-level danger	DB unreachable, disk full

! WATCH OUT — Never log secrets or full payloads

Tokens, passwords, full email bodies, payment details — log *that* they exist, never their contents. A log line that helps you debug today is a leak tomorrow when someone pulls the logs into a less-secured tool. Mask aggressively at the source.

Wiring uvicorn's logs through structlog

By default uvicorn emits its own text logs. Run it with structured JSON so everything in your pipeline matches.

```
fastapi run app/main.py --no-access-log
# then configure access logs yourself via the middleware above
# (avoids two formats fighting in your log stream)
```

TRY IT YOURSELF — Make PulseChat's logs structured

Add `configure_logging()` to the lifespan, the logging-context middleware, and bind `user_id` inside the auth dependency. Hit a few endpoints and watch the JSON lines — every line in one request shares the same `request_id`.

Testing (pytest, TestClient, mocking)

A real API earns trust through a real test suite. FastAPI makes this easier than most frameworks:

`TestClient` runs your app in-process, dependency overrides let you swap real services for fakes, and pytest fixtures make the whole thing composable.

Install the testing stack

```
pip install pytest httpx pytest-asyncio
# httpx is what TestClient is built on; required for async tests too
```

Your first test: TestClient

A API TERM — TestClient

A FastAPI-provided client that runs your app in the same process — no real network, no spawned server. You call endpoints with familiar `.get()`, `.post()` methods and get a response object to assert against. Behind the scenes it uses `httpx` to drive your app's ASGI interface directly.

tests/test_health.py

```
# tests/test_health.py
from fastapi.testclient import TestClient
from app.main import app

def test_ping():
    with TestClient(app) as client:      # the `with` triggers lifespan
        r = client.get("/ping")
        assert r.status_code == 200
        assert r.json() == {"status": "ok"}
```

★ PRO TIP — Use the with statement

`TestClient(app)` alone does not run the lifespan. Wrap it in `with` and startup/shutdown fire correctly — important the moment your app needs Redis or a DB pool. The official testing docs lead with this pattern.

Pytest fixtures: setup you reuse

Stop repeating "spin up a test client" in every test. Put it in a fixture.

tests/conftest.py

```
# tests/conftest.py
import pytest
```



```
from fastapi.testclient import TestClient
from app.main import app
```

```
@pytest.fixture
def client():
    with TestClient(app) as c:
        yield c
```

```
def test_ping(client):
    assert client.get("/ping").status_code == 200
```

Mocking dependencies — the FastAPI superpower

Real tests must not touch the production database, send real emails, or hit real third-party APIs. FastAPI's

`dependency_overrides` swaps any dependency for a fake.

```
# tests/conftest.py
import pytest
from app.main import app
from app.dependencies import get_current_user
from app.db.session import get_db

class FakeUser:
    id, email, is_active = 1, "test@example.com", True

@pytest.fixture
def fake_user():
    return FakeUser()

@pytest.fixture(autouse=True)
def override_user(fake_user):
    app.dependency_overrides[get_current_user] = lambda: fake_user
    yield
    app.dependency_overrides.clear()
```

Now every test runs as the fake logged-in user — no JWT, no password hashing, no DB lookup. The same trick works for the database session, an HTTP client, a Redis connection.

A real test database with a fixture

Override `get_db` to point at a separate SQLite test database, create the tables for the test, drop them after.

```
# tests/conftest.py
from sqlalchemy.ext.asyncio import create_async_engine, async_sessionmaker
from app.db.base import Base
from app.db.session import get_db
```

```

@pytest.fixture
async def test_db():
    engine = create_async_engine("sqlite+aiosqlite:///memory:")
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
    SessionLocal = async_sessionmaker(engine, expire_on_commit=False)
    async with SessionLocal() as session:
        yield session
    await engine.dispose()

@pytest.fixture
def client(test_db):
    async def _get_db():
        yield test_db
    app.dependency_overrides[get_db] = _get_db
    with TestClient(app) as c:
        yield c
    app.dependency_overrides.clear()

```

A API TERM — `dependency_overrides`

A dictionary on the `FastAPI` instance that maps real dependency callables to fake ones. FastAPI checks it first when resolving any `Depends()`. Always clear it after the test or overrides bleed between tests.

Testing CRUD endpoints

```

def test_create_message(client):
    resp = client.post("/messages",
                      json={"channel": "general", "content": "hi"})
    assert resp.status_code == 201
    body = resp.json()
    assert body["content"] == "hi"
    assert body["id"] > 0

def test_validation_rejects_blank(client):
    resp = client.post("/messages",
                      json={"channel": "general", "content": " "})
    assert resp.status_code == 422
    assert resp.json()["fields"][0]["field"] == "content"

```

Testing WebSockets

`TestClient` handles WebSockets natively — synchronous, easy to read.

```

def test_chat_broadcast(client):
    with client.websocket_connect("/ws/chat/general") as ws_a, \
         client.websocket_connect("/ws/chat/general") as ws_b:
        ws_a.send_json({"content": "hello"})

```

```
msg = ws_b.receive_json()
assert msg["content"] == "hello"
```

Mocking Celery tasks

You don't want tests to spawn real worker processes. Use Celery's eager mode (tasks run inline) or simply patch `.delay` to a no-op.

```
from unittest.mock import patch

def test_message_enqueues_fanout(client):
    with patch("app.worker.tasks.fanout_push.delay") as fake:
        client.post("/messages", json={"channel": "general", "content": "hi"})
        fake.assert_called_once()
```

Async tests with httpx

For genuinely async test paths (e.g. calling code that awaits external services), drop `TestClient` and use `httpx.AsyncClient` with FastAPI's `ASGITransport`.

```
import pytest, httpx
from httpx import ASGITransport
from app.main import app

@pytest.mark.asyncio
async def test_ping_async():
    transport = ASGITransport(app=app)
    async with httpx.AsyncClient(transport=transport,
                                base_url="http://test") as client:
        async with app.router.lifespan_context(app):
            r = await client.get("/ping")
            assert r.status_code == 200
```

What to test, and how much

Test type	Aim	Examples
Happy path	The normal flow returns success	POST /messages with good body returns 201
Validation	Bad input is rejected with 422	blank content, missing field, oversized
Auth	Protected routes 401 without token	GET /me without Authorization header
Permissions	User cannot touch others' data	User A can't read user B's tasks
Edge cases	Boundaries, empties, duplicates	empty list, max-length, repeat post
Integration	Two pieces work together	

Test type	Aim	Examples
		POST /messages triggers Celery enqueue

★ **PRO TIP — Coverage is a compass, not a destination**

Aim for tests on every endpoint's happy path plus the most likely failures. 100% line coverage on a system that doesn't test its real behaviour is worse than 60% on the right behaviours. Test what users would notice, not what your tool can count.

▶ **TRY IT YOURSELF — Cover one PulseChat feature end to end**

Pick the message-posting flow. Write tests for: success, blank content rejected, unauthenticated rejected, fan-out task is enqueued, broadcast is received via WebSocket. Five tests, one feature, full confidence.

Monitoring (Prometheus & Sentry)

Once your API is live, you need two senses: a heartbeat (is everything healthy?) and an alarm (something just broke). **Prometheus** provides the heartbeat — metrics scraped continuously. **Sentry** provides the alarm — exceptions captured and grouped the moment they happen.

The two roles, side by side

	Prometheus (metrics)	Sentry (errors)
Answers	How is the service behaving overall?	What exactly broke, and where?
Granularity	Aggregated numbers over time	Individual exception with stack trace
Trigger	You pull (scrape) periodically	Pushed the instant an error occurs
Use to	Spot trends, alert on rates	Debug a specific failure
Dashboards	Grafana	Sentry UI

You want both. A 95th-percentile latency spike (Prometheus) draws your attention; the exception that caused it (Sentry) tells you the line of code.

Prometheus: emit metrics

A API TERM — Prometheus & the four key metrics

Prometheus is a pull-based metrics system: your app exposes `/metrics` as plain text and a Prometheus server scrapes it every few seconds. The four metric types you will use are **Counter** (only goes up — request count), **Gauge** (up and down — active WebSocket connections), **Histogram** (distributions — request latency), and **Summary** (similar to Histogram, computed client-side).

The simplest path: `prometheus-fastapi-instrumentator` auto-instruments every endpoint with sensible defaults.

```
pip install prometheus-fastapi-instrumentator
```

```
# app/main.py
from prometheus_fastapi_instrumentator import Instrumentator

app = FastAPI(title="PulseChat API", lifespan=lifespan)

Instrumentator().instrument(app).expose(app, endpoint="/metrics")
```

That single block gives you request counts, latency histograms, in-flight counts, and status-code breakdowns, all available at `/metrics`.

Custom metrics for your domain

Auto-metrics are great for HTTP. For domain events (messages sent, WebSocket connects, Celery tasks completed) define your own.

```
from prometheus_client import Counter, Gauge, Histogram

messages_posted = Counter(
    "pulse_messages_posted_total",
    "Total messages posted",
    labelnames=("channel",),
)

active_ws = Gauge(
    "pulse_websocket_active",
    "Currently open WebSocket connections",
)

celery_task_duration = Histogram(
    "pulse_celery_task_seconds",
    "Time spent in Celery tasks",
    labelnames=("task_name",),
)

# In your endpoints / handlers:
messages_posted.labels(channel=data.channel).inc()
active_ws.inc(); ... active_ws.dec()
```

★ PRO TIP — Cardinality discipline

Labels multiply: ten channels times five statuses = fifty time series. Never put unbounded values in labels — user ids, message ids, IPs. Each unique label combination is its own time series and high-cardinality metrics will crush a Prometheus server. Keep labels to small, known sets (channel name, route, method, status code).

Visualising with Grafana

Grafana is the standard dashboard tool. Point it at your Prometheus server, then build panels: request rate per minute, p95 latency, active WebSockets, error rate per route. The PromQL queries are short —

`rate(http_requests_total[5m])`, `histogram_quantile(0.95, ...)`. Save the dashboard, share it with the team, and you suddenly have a live picture of your system.

Sentry: catch every exception

A API TERM — Sentry

An error-tracking platform: your app sends every uncaught exception (and optionally warnings, performance traces, breadcrumbs) to Sentry, which groups duplicates, deduplicates noise, and notifies you. The first time a new error appears in production, you know.

```
pip install "sentry-sdk[fastapi]"
```

```
# app/core/observability.py
import sentry_sdk
from sentry_sdk.integrations.fastapi import FastApiIntegration
from sentry_sdk.integrations.sqlalchemy import SQLAlchemyIntegration
from sentry_sdk.integrations.celery import CeleryIntegration
from app.core.config import get_settings

def init_sentry() -> None:
    settings = get_settings()
    if not settings.SENTRY_DSN:
        return # opt-in
    sentry_sdk.init(
        dsn=settings.SENTRY_DSN,
        environment=settings.ENV, # "prod" / "staging"
        release=settings.RELEASE, # git sha
        traces_sample_rate=0.1, # sample 10% of traces
        send_default_pii=False, # privacy-first
        integrations=[
            FastApiIntegration(),
            SQLAlchemyIntegration(),
            CeleryIntegration(),
        ],
    )
```

Initialise from your lifespan (so it picks up settings) and forget about it. Any exception your app raises is now captured with the full stack, request context, and user id (if you configured PII to be included).

Tying it all together with request id

The request id from Phase 2's middleware is the glue. Attach it as a Sentry tag and include it in every log line (Chapter 7). When a user reports a problem, ask for the request id from their network tab — and use it to jump between logs, traces, and the Sentry issue.

```
import sentry_sdk

@app.middleware("http")
async def attach_request_id(request, call_next):
    rid = request.state.request_id
    with sentry_sdk.configure_scope() as scope:
```

```
scope.set_tag("request_id", rid)
return await call_next(request)
```

What to alert on

Signal	Reasonable alert threshold
HTTP 5xx rate	> 1% of requests for 5 minutes
p95 latency	> 2x normal baseline for 10 minutes
Celery queue depth	> 1000 pending for 5 minutes
Redis memory	> 80% capacity
New Sentry issue	Immediate — any first occurrence
WebSocket disconnect rate	Sudden spike (network problem)

! WATCH OUT — Alert fatigue is real

Every false alarm trains the team to ignore alerts. Start with a small number of high-confidence rules and tune from there. 'Wake me up' alerts should require human action; everything else is a dashboard, not a pager.

🎮 TRY IT YOURSELF — Make PulseChat observable

Add prometheus-fastapi-instrumentator, one custom `messages_posted` counter and one `active_ws` gauge, and Sentry init in the lifespan. Hit `/metrics` and confirm your counter ticks up. Then deliberately raise an exception from a test endpoint and watch it appear in Sentry within seconds.

The Complete PulseChat API

Everything from Phase 3, assembled into one service. Multiple processes cooperating: the API, a Celery worker, a Celery Beat scheduler, all backed by Redis and Postgres. This is the shape of a production Python web service in 2026.

The full layout

Final project layout

```
pulsechat/
├── alembic/                # migrations (Phase 2, Ch. 6)
├── docker-compose.yml      # Postgres + Redis for development
├── .env                   # SECRET_KEY, DATABASE_URL, REDIS_URL, SENTRY_DSN
├── tests/
│   ├── conftest.py        # client + overrides (Ch. 8)
│   ├── test_messages.py
│   └── test_websocket.py
└── app/
    ├── main.py            # FastAPI() + lifespan + observability
    ├── core/
    │   ├── config.py      # pydantic-settings (Ch. 1)
    │   ├── logging.py     # structlog setup (Ch. 7)
    │   ├── limiter.py     # slowapi + Redis (Ch. 4)
    │   └── observability.py # Sentry + Prometheus (Ch. 9)
    ├── db/
    │   ├── session.py     # async SQLAlchemy
    │   └── redis.py       # get_redis dependency (Ch. 5)
    ├── ws/
    │   └── hub.py          # ChatHub w/ Redis pub-sub (Ch. 2, 5)
    ├── worker/
    │   ├── celery_app.py  # Celery config (Ch. 6)
    │   └── tasks.py       # send_email, fanout_push (Ch. 6)
    ├── models/            # SQLAlchemy ORM
    ├── schemas/           # Pydantic v2 w/ validators & computed (Ch. 1)
    ├── crud/
    ├── routers/
    │   ├── auth.py
    │   ├── messages.py
    │   └── channels.py
    ├── ws_routes.py       # /ws/chat/* WebSocket endpoints
    └── dependencies.py
```

app/main.py — the integration point

app/main.py

```
import asyncio
from contextlib import asynccontextmanager
from fastapi import FastAPI, Request
```

```

from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse
from redis import asyncio as aioredis
from slowapi.errors import RateLimitExceeded
from slowapi.middleware import SlowAPIMiddleware
from prometheus_fastapi_instrumentator import Instrumentator
import structlog, time, uuid

from app.core.config import get_settings
from app.core.logging import configure_logging
from app.core.observability import init_sentry
from app.core.limiter import limiter
from app.ws.hub import ChatHub
from app.routers import auth, messages, channels
from app import ws_routes

log = structlog.get_logger()

@asynccontextmanager
async def lifespan(app: FastAPI):
    configure_logging("INFO")
    init_sentry()
    settings = get_settings()
    app.state.redis = aioredis.from_url(settings.REDIS_URL,
                                       decode_responses=True,
                                       max_connections=20)

    app.state.chat_hub = ChatHub(app.state.redis)
    subscriber = asyncio.create_task(app.state.chat_hub.listen_forever())
    log.info("startup_complete")
    try:
        yield
    finally:
        subscriber.cancel()
        try: await subscriber
        except asyncio.CancelledError: pass
        await app.state.redis.aclose()
        log.info("shutdown_complete")

app = FastAPI(title="PulseChat API", version="1.0.0", lifespan=lifespan)
app.state.limiter = limiter

# ---- middleware (CORS last so its headers wrap everything else) -----
@app.middleware("http")
async def request_context(request: Request, call_next):
    rid = request.headers.get("X-Request-ID", str(uuid.uuid4()))
    request.state.request_id = rid
    structlog.contextvars.clear_contextvars()
    structlog.contextvars.bind_contextvars(
        request_id=rid, method=request.method, path=request.url.path,
    )
    start = time.perf_counter()
    log.info("request_started")
    try:
        response = await call_next(request)

```

```

        return response
    finally:
        elapsed_ms = (time.perf_counter() - start) * 1000
        try:
            response.headers["X-Request-ID"] = rid
            response.headers["X-Process-Time-ms"] = f"{elapsed_ms:.1f}"
            log.info("request_finished",
                    status=response.status_code, ms=round(elapsed_ms, 1))
        except UnboundLocalError:
            pass

app.add_middleware(SlowAPIMiddleware)
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

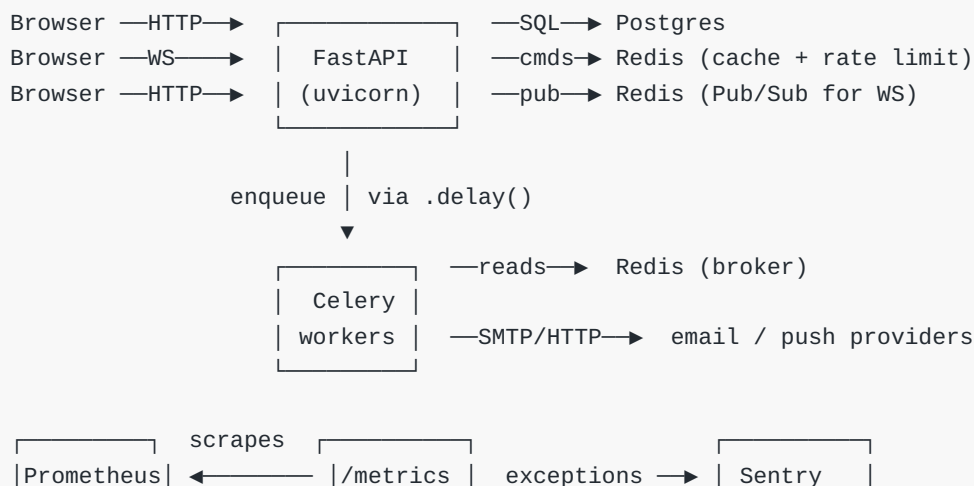
# ---- error handling ----
@app.exception_handler(RateLimitExceeded)
async def _rl(request: Request, exc: RateLimitExceeded):
    return JSONResponse(429,
        content={"error": "rate_limited", "detail": str(exc.detail),
                "request_id": request.state.request_id},
        headers={"Retry-After": str(exc.retry_after or 60)})

# ---- observability ----
Instrumentator().instrument(app).expose(app, endpoint="/metrics")

# ---- routes ----
app.include_router(auth.router)
app.include_router(channels.router)
app.include_router(messages.router)
app.include_router(ws_routes.router)

```

What is now running (the full picture)



↓
Grafana dashboards & alerts

Running it locally

```
# Terminal 1 - API
fastapi dev app/main.py

# Terminal 2 - Celery worker
celery -A app.worker.celery_app worker --loglevel=info

# Terminal 3 - Celery Beat (scheduled tasks)
celery -A app.worker.celery_app beat --loglevel=info

# Terminal 4 - tests
pytest -q
```

What you can now do

- ✓ Express complex business rules in Pydantic with validators and computed fields
- ✓ Build real-time features with WebSockets, including multi-instance broadcasting via Redis
- ✓ Use lifespan to open and cleanly close every long-lived resource
- ✓ Defend endpoints with per-route rate limits backed by a shared Redis store
- ✓ Use Redis for both caching and Pub/Sub coordination across processes
- ✓ Offload durable, retryable work to Celery workers with optional scheduling
- ✓ Emit structured, contextual logs that survive distributed debugging
- ✓ Cover your API with pytest, TestClient, dependency overrides, and mocked tasks
- ✓ Monitor health with Prometheus metrics and catch exceptions with Sentry

Glossary of Advanced Terms

Every new concept from Phase 3 in one place.

Term	Meaning
Beat (Celery)	A scheduler process that enqueues tasks at intervals or on a cron schedule; run exactly one.
Bind (Celery task)	<code>bind=True</code> makes the task receive its own instance as the first argument, enabling <code>self.retry()</code> .
Broker	The message bus (Redis or RabbitMQ) Celery uses to deliver jobs from the API to workers.
Cache invalidation	Removing or updating cached data when the underlying source changes — famously one of the hardest problems in software.
Cardinality (metrics)	The number of unique label combinations a metric produces; keep it bounded or your Prometheus suffers.
Celery	The standard Python task queue: define tasks, call <code>.delay</code> , workers run them durably.
Close code (WS)	A numeric reason a WebSocket was closed (1000 normal, 1008 policy, 1011 server error).
Computed field	A read-only field in a Pydantic model produced by a method, decorated with <code>@computed_field</code> .
Counter (Prometheus)	A monotonic metric that only increases — request totals, errors.
dependency_overrides	FastAPI dictionary that replaces real dependencies with fakes during tests.
Eager mode (Celery)	Test mode where <code>.delay</code> runs the task inline instead of through the broker.
Field validator	A method decorated <code>@field_validator</code> that runs custom checks/transforms on a Pydantic field.
Gauge	A Prometheus metric that goes up and down — active connections, queue depth.
Histogram	A Prometheus metric that records value distributions — request latency, payload sizes.
Lifespan	Async context manager passed to <code>FastAPI(lifespan=...)</code> for startup and shutdown logic.
Model validator	A method decorated <code>@model_validator(mode='after')</code> that runs checks spanning multiple fields.
on_event (legacy)	

Term	Meaning
	The deprecated <code>@app.on_event("startup"/"shutdown")</code> decorators; replaced by lifespan.
Prometheus	A pull-based metrics system: your app exposes <code>/metrics</code> ; Prometheus scrapes periodically.
pwdlib / passlib	Password hashing libraries — pwdlib is the modern recommendation in current FastAPI docs.
Pub/Sub (Redis)	Publish-subscribe messaging: one publisher, many subscribers; used to broadcast across server instances.
pytest fixture	A reusable setup function for tests, decorated <code>@pytest.fixture</code> ; injected by name.
Rate limiting	Capping how many actions a client may take in a window; returns HTTP 429 when exceeded.
Read-through cache	Pattern where reads first check the cache, fall through to the source, then populate the cache.
redis.asyncio	The async API of redis-py; replaces the deprecated standalone aioredis package.
Result backend	Optional Celery storage that retains task results for later querying.
Retry-After (header)	HTTP header on a 429 telling clients how many seconds to wait before retrying.
Sentry	An error-tracking service that captures exceptions with stack trace and request context.
slowapi	FastAPI-friendly rate-limiting library built on the <code>limits</code> package.
Structured logging	Emitting log records as JSON with consistent fields, queryable by log tools.
structlog	A structured logging library with first-class context binding — pairs well with FastAPI's per-request scope.
Task acks_late	Celery option that acknowledges a job only after success, so a worker crash re-queues it.
TestClient	FastAPI's in-process test client built on httpx; supports HTTP and WebSocket.
Token bucket	The classic rate-limiting algorithm: tokens refill at a rate, each request takes one; out of tokens = blocked.
Traces (Sentry)	Optional performance traces that record the time spent in different parts of a request.
WebSocket	A long-lived, bidirectional protocol over a single TCP connection, used for real-time features.

What Comes Next

PulseChat now behaves like a real production service: real-time, distributed, observable, and tested. Phase 4 is about putting it in front of real users at scale — packaging it, deploying it, hardening it, versioning it, and tuning it.

COMING IN PHASE 4 · PROFESSIONAL

● Shipping & running it for real

The final phase takes your skills from 'I can build this' to 'I can run this in production for many users.' Same project, taken from your laptop to a real deployment.

Phase 4 topics

- **Docker & Deployment** — Dockerfiles, Compose, gunicorn + uvicorn workers
- **Security Best Practices** — HTTPS, secret management, OWASP API Top-10
- **API Versioning & Design** — URL vs header versioning, deprecation policy
- **OpenAPI Schema & SDK Generation** — auto-generate clients in any language
- **Performance Tuning** — profiling, connection pooling, async I/O patterns
- **Microservices with FastAPI** — splitting a monolith, internal HTTP & queues
- **CI/CD for FastAPI** — automated tests, container builds, safe rollouts

Before you move on — solidify Phase 3

- ✓ Rebuild the multi-instance chat broadcast from scratch; explain to yourself why Redis Pub/Sub is needed
- ✓ Add a Pydantic `@model_validator` that enforces a cross-field rule and add a test for it
- ✓ Write a Celery task with retry and prove (with logs) that it really retries on a forced failure
- ✓ Tune a custom Prometheus metric and design two Grafana panels you would actually look at
- ✓ Use a request id to trace one failing request from Sentry back to its structured log line

i NOTE — You are now an advanced FastAPI developer

Async, real-time, distributed coordination, durable background work, observability, and tests are the skills that separate weekend projects from systems people depend on. Phase 4 turns these into a deployable, maintainable product.

End of Phase 3. Verified against the official FastAPI documentation at fastapi.tiangolo.com — 2026 edition. FastAPI 0.132+, Python 3.10+, Pydantic v2, redis-py 5.x (with `redis.asyncio`), Celery 5.x, structlog, slowapi.